

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
K K BIRLA GOA CAMPUS

2024-25 Sem. I	Mid-Sem Regular	Open Book / Laptop
Principles of Programming Languages	CS F301	35%=70 marks; Pages=2;
5 Oct 2024	90 min	Questions=7 1130-1300 hrs

For all questions: Study the two program modules[⊙] `Module1.c` and `Module2.c` given on the last page and answer the following questions.

Q1. Will each module `Module?.c` compile independently (by “`gcc -c Module?.c`” or in two separate commands “`gcc -c Module1.c ; gcc -c Module2.c`”)? Justify your answer: why if not, how each is processed if yes. [Marks 10]

Q2. Will `Module1.c` alone yield an executable by “`gcc Module1.c`”? Will the two modules (compiled to object files) link *together* and yield an executable? Justify: why not, or how indeed. [Marks 10]

Q3. (a) What number replaces `ClassSize`? What is the returned value of `main`? You can give these numbers in terms of the number of bytes, or a multiple of integer size, etc.; but it is necessary to mention the unit you are using. [Marks 5]
(b) To fit the programs on one page we compressed many unnecessary spaces. Can you express one single rule in one sentence for when a space is needed in C-source? [Marks 5]

Q4. Which lines in each module will produce imperative action code[⊕] in translation? Give the line numbers modulewise. Justify: why only these lines? [Marks 10]

Q5. Classify all the identifiers (type names except keywords, variable and member names, function names) by their binding times—that means: compile time, procedure activation time, linking time, loading time, run time. Also indicate which identifiers will be seen in the assembly output after compilation. [Marks 10]

Q6. If we change the ending “`return 0;`” in both `f` and `g` to “`return 1;`”, what difference will it make to the running of the program (with both modules linked together), if any? Justify and explain your answer thoroughly. [Marks 10]

Q7. Demonstrate with justification how to calculate the maximum total stack frame size beginning with the root call `f(n)` in terms of `n` and any combination of `sizeof(char)`, `sizeof(int)`, `sizeof(char*)` etc.. [Marks 10]

Module1.c

```

1 #include<malloc.h>
2 struct Abstract;
3 typedef struct Abstract *Class;
4 extern void Subroutine1(Class, const char *, int);
5 extern void Subroutine2(Class, const char *, int);
6 struct Abstract{
7     void *abstractData;
8     Class (*Abstract)(void *abstractData);
9     void (**abstractMethod)(Class this, ...);
10 };
11 #define ClassSize sizeof(struct Abstract)
12 Class object1;
13 int main(void) {
14     object1 = (Class)malloc(ClassSize);
15     object1->abstractData = malloc(256);
16     object1->abstractMethod = (void (**)(Class, ...)) malloc(2*sizeof(void
        (*) (Class, ...)));
17     object1->abstractMethod[0] = ((void(*) (Class, ...))&Subroutine1);
18     object1->abstractMethod[1] = (void(*) (Class, ...))&Subroutine2;
19     object1->abstractMethod[0](object1, "", 0);
20     object1->abstractMethod[1](object1, "", 0);
21     Class object2 = object1;
22     Class object1 = object2;
23     if(f(10)) {
24         Class object1 = object2;
25         object2->Abstract = (Class*)(void*)&Subroutine1;
26         if(g(10)) {
27             Class object2 = object1;
28             Class object1 = object2;
29         }
30     }
31     object1 = object1->Abstract(object1);
32     object2 = object1->Abstract(object2);
33     return sizeof(object1);
34 }

```

Module2.c

```

1 struct Abstract;
2 typedef struct Abstract *Class;
3 void Subroutine1(Class c, const char *s, int i) { return; }
4 void Subroutine2(Class c, const char *s, float f) { return; }
5 int f(int n){ if(n>0)return f(n-1)>g(n-1)?g(n-2):f(n-2);return 0;}
6 int g(int n){ if(n>0)return g(n-1)<f(n-1)?f(n-2):g(n-2);return 0;}

```

NB:⊙ None of the modules have syntax errors in punctuation, e.g. the function pointer notation *type (*name)(...)* and function pointer array notation *type (**name)(...)*, the *printf*-like variadic signature *void (**abstractMethod)(Class this, ...)*; etc. are all fine. So do not smell a rat there. The questions are about type systems, modular structure and procedure activation. ⊕ Imperative code means code that is run in the course of a program's execution—if and when control reaches that code. e.g. Expression calculation like “*a=b+c;*”.

SOLUTION

A1. Each module compiles independently (not linking, only compilation with `-c` or `-S`). Except for implicit prototypes all the type information needed for compilation (static) is available to each module. The following is the transcript of the process.

```
rs-joshi@PoPL$ for name in `ls Module?.c` ; do echo "Compiling" $name ; gcc -c -Wall $name ; done
Compiling Module1.c
Module1.c: In function 'main':
Module1.c:23:12: warning: implicit declaration of function 'f' [-Wimplicit-function-declaration]
   23 |         if(f(10)) {
       |            ^
Module1.c:26:20: warning: implicit declaration of function 'g' [-Wimplicit-function-declaration]
   26 |             if(g(10)) {
       |                ^
Module1.c:28:31: warning: unused variable 'object1' [-Wunused-variable]
   28 |                 Class object1 = object2;
       |                             ~~~~~~
Compiling Module2.c
Module2.c: In function 'f':
Module2.c:5:35: warning: implicit declaration of function 'g' [-Wimplicit-function-declaration]
    5 | int f(int n){if(n>0)return f(n-1)>g(n-1)?g(n-2):f(n-2);return 0;}
       |                                   ^
```

Because `Module2.c` doesn't have any instantiation of the type `struct Abstract`, it needs only to know that `Class` is a pointer to that type. As long as all pointers are of the same size and layout, it doesn't require the type just to pass on a pointer. The type's storage information—size and layout—is needed only when a pointer is dereferenced or the type *value* is to be passed around instead of the pointer.

A2. No, `Module1.c` alone cannot yield an executable unless bodies of functions `f`, `g`, `Subroutine1`, `Subroutine2` are given, thus, alone, the linker will give four errors. But the two modules together are a complete program, in object form as well, and yield an executable after linking the object files.

Those who cited a particular (ISO) standard (and/or made assumptions about auxiliary code such as headers not shown in the question paper) may or may not get credit. Their dependence on authority and neglect of obvious practical considerations (that we have witnessed quite often in the tutorial sessions as well as other programming labs) reveals their lack of grasp of very basic principles of programming languages, like the ones on the slide “Higher Level Languages as Abstractions”; in particular, that expression and translation of the computation intended by the programmer is an essential requirement. Our [⊙] instruction also was to tell you to not look for syntax errors but look at the type systems, modularity and procedure activation. Therefore, how many marks the sticklers for syntactic prudishness will get for this question depends on whether they have tried to go beyond that in their comprehension or not. And this will be assessed by the subjective judgement of the IC.

A3. (a) `ClassSize = 3*sizeof(void*)` assuming all pointer sizes to be the same. And `main` returns `sizeof(void*)`.

(b) The single necessary rule is: after each (1)identifier and (2)literal constant (3)that does not end in a (4)non-space delimiter (like quotes), we need some whitespace as a delimiter.

A4. Line number list:

Module1.c 13-28; 31-33. **Module2.c** 3-6. Justification: Type declarations, static (compile time) object definitions do not produce imperative code.

A5. Table:

Identifier	Declaration	Binding Time
Abstract	Module1 Lines 2,6,8 Module2 Line 1	Compile time
Class	Module1 Line 3 Module2 Line 2	Compile time
abstractData	Module1 Line 7	Compile time
	Module1 Line 8	Procedure Activation Time
abstractMethod	Module1 Line 9	Compile time
this	Module1 Line 9	Procedure Activation time
ClassSize	Module1 Line 11	Preprocessing time
object1	Module1 Line 12	Compile time
	Module 1 Lines 22,24,28	Procedure Activation time
object2	Module1 Lines 21,27	Procedure Activation time
Subroutine1	Module1 Line 4	Linking time
	Module2 Line 3	
Subroutine2	Module1 Line 5	Linking time
	Module2 Line 4	
c,s,i,n	Module2 Lines 3,4,5,6	Procedure Activation time
f	Module2 Line 4	Procedure Activation time
	Module2 Line 5	Linking time
g	Module2 Line 6	Linking time

A6. In the current program, we will get a segfault in line 31 because the functions `f,g` return 0, the two nested `if` blocks are not executed, hence `object1`'s `Abstract` function pointer member is uninitialised, thus a call is placed to an invalid text section address. When we change all "`return 0;`" to "`return 1;`", both the nested `if` blocks do get executed, and the program executes to return the size of a pointer.

A7. The max number of stackframes of `f` and `g` together are $n + 1$; both functions have the same stackframe size; the total size is

$$(n + 1) * (\text{sizeof}(\text{int}) + 3 * \text{sizeof}(\text{void*}))$$

because one integer paramter and three addresses (return address, stack pointer, stack frame base pointer) are to be saved.